



Parallel DBMS



OUTLINE

- ✓ Parallel Computer and Parallel Database System
- ✓ Functional Aspects Provided by Parallel Database Systems
- ✓ Parallel Architectures
- ✓ Hierarchical Architecture
- ✓ Components of Parallel DBM Architecture.
- ✓ Data Partitioning Techniques
- ✓ Parallel Query Optimization
- ✓ Database Integration Process



Parallel DBMS

❑ Parallel Computer

➤ A parallel computer, or **multiprocessor**, is itself a distributed system made of a number of nodes (processors and memories) **connected by a fast network within a cabinet.**

❑ Parallel Database System

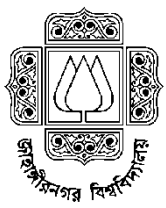
➤ Parallel database systems combine **database management** and **parallel processing** in order to deliver high-performance and high-availability database servers at a much lower price than equivalent mainframe computers.



Parallel DBMS

□ Parallel Processing

- Parallel processing divides a large task into many smaller tasks, and executes the smaller tasks concurrently on several nodes. As a result, the larger task completes more quickly.
- A *node* is a separate processor, often on a separate machine. Multiple processors, however, can reside on a single machine.
- Some tasks can be effectively divided, and thus are good candidates for parallel processing. Other tasks, however, do not lend themselves to this approach.



Parallel DBMS

□ Parallel Processing

➤ For example, in a bank with only one teller, all customers must form a single queue to be served. With two tellers, the task can be effectively split so that customers form two queues and are served twice as fast-or they can form a single queue to provide fairness. This is an instance in which parallel processing is an effective solution.

➤ By contrast, if the bank manager must approve all loan requests, parallel processing will not necessarily speed up the flow of loans. No matter how many tellers are available to process loans, all the requests must form a single queue for bank manager approval. No amount of parallel processing can overcome this built-in bottleneck to the system.



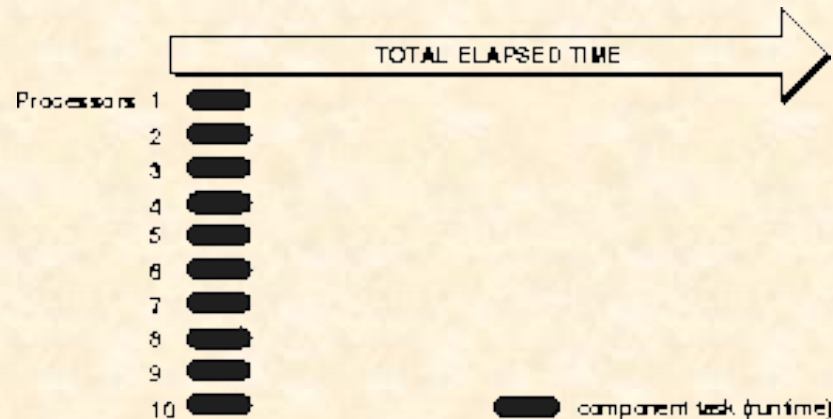
Parallel DBMS

□ Parallel Processing

➤ Sequential Processing of a Large Task



➤ Parallel Processing (Executing Component Tasks in Parallel)





Parallel DBMS

□ Parallel Processing

- In sequential processing, the query is executed as a single large task.
- In parallel processing, the query is divided into multiple smaller tasks, and each component task is executed on a separate node.



Parallel DBMS

❑ Problems of Parallel Processing

➤ Effective implementation of parallel processing involves two challenges:

- ✓ Structuring tasks so that certain tasks can execute at the same time (in parallel)
- ✓ Preserving the sequencing of tasks which must be executed serially

❑ Characteristics of a Parallel System

- A parallel processing system has the following characteristics:
- ✓ Each processor can perform tasks concurrently.
 - ✓ Tasks may need to be synchronized.
 - ✓ Nodes usually share resources, such as data, disks, and other devices.



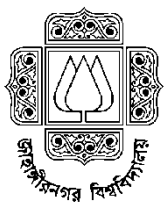
Parallel DBMS

❑ Key Elements of Parallel Processing

- Speedup
- Scaleup
- Synchronization
- Locking
- Messaging

❑ Speedup

❑ Scaleup



Parallel DBMS

❑ Synchronization: A Critical Success Factor

- ✓ Coordination of concurrent tasks is called *synchronization*. It is necessary for correctness.
- ✓ The key to successful parallel processing is to divide up tasks so that very little synchronization is necessary.
- ✓ In parallel processing between nodes, a high-speed interconnect is required among the parallel processors.
- ✓ The overhead of this synchronization can be very expensive if a great deal of inter-node communication is necessary.
- ✓ The amount of synchronization depends on the amount of resources and the number of users and tasks working on the resources. Little synchronization may be needed to coordinate a small number of concurrent tasks, but lots of synchronization may be necessary to coordinate many concurrent tasks.



Parallel DBMS

❑ Synchronization: A Critical Success Factor

- ✓ A great deal of time spent in synchronization indicates high contention for resources.
- ✓ Too much time spent in synchronization can diminish the benefits of parallel processing. With less time spent in synchronization, better speedup and scaleup can be achieved.



Parallel DBMS

❑ Locking

- ✓ Locks are fundamentally a way of synchronizing tasks. Many different locking mechanisms are necessary to enable the synchronization of tasks required by parallel processing.
- ✓ The Integrated Distributed Lock Manager (Integrated DLM, or IDLM) is the internal locking facility used with Oracle Parallel Server. It coordinates resource sharing between nodes running a parallel server. Each node operates independently of other nodes, except when contending for the same resource.
- ✓ The IDLM performs the following services for applications:
 - Keeps track of the current "ownership" of a resource
 - Accepts lock requests for resources from application processes
 - Notifies the requesting process when a lock on a resource is available
 - Gets access to a resource for a process



Parallel DBMS

□ Messaging

- ✓ Parallel processing requires fast and efficient communication between nodes: a system with high bandwidth and low latency which efficiently communicates with the IDLM.
- ✓ *Bandwidth* is the total size of messages which can be sent per second.
- ✓ *Latency* is the time (in seconds) it takes to place a message on the interconnect.



Parallel DBMS

□ Parallel DBMS: Definition

- ✓ Parallel DBMS is a Database Management System that runs through multiple processors and disks.
- ✓ They combine two or more processors and also disk storage that helps make operations and executions easier and faster.
- ✓ They are designed to execute concurrent operations.

□ Why Parallel DB? (Motivation for Parallel DB)

- ✓ Nowadays organizations need to handle a huge amount of data with a high transfer rate. For such requirements, the client-server or centralized system is not efficient. With the need to improve the efficiency of the system, the concept of the parallel database comes in picture.



Parallel DBMS

□ Why Parallel DB? (Motivation for Parallel DB)

- ✓ Parallel machines are becoming quite common and affordable. Prices of microprocessors, memory and disks have dropped sharply.
- ✓ Databases are growing increasingly large.
- ✓ Large volumes of transaction data are collected and stored for later analysis.
- ✓ Multimedia objects like images are increasingly stored in databases
- ✓ A parallel database system seeks to improve the performance of the system through parallelizing concept.
- ✓ Multiple resources like CPUs and Disks are used in parallel.
- ✓ The operations are performed simultaneously, as opposed to serial processing.
- ✓ It performs many parallelization operations like data loading, query processing, building indices, and evaluating queries.



Parallel DBMS

❑ Functional Aspects Provided by Parallel Database Systems

Ideally, a parallel database system should have the following functional aspects.

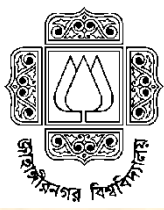
- **Performance Improvement:** By connecting multiple resources like CPU and disks in parallel we can significantly increase the performance of the system.
- **High-availability:** In the parallel database, nodes have less contact with each other, so the failure of one node in the cluster doesn't cause for failure of the entire system. This amounts to significantly higher database availability.



Parallel DBMS

❑ Functional Aspects Provided by Parallel Database Systems

- **Proper resource utilization:** Due to parallel execution, the CPU will never be idle. Thus, proper utilization of resources is there.
- **Increase Reliability:** When one site fails, the execution can continue with another available site in the cluster which is having a copy of data. Making the system more reliable.



Parallel DBMS

❑ Functional Aspects Provided by Parallel Database Systems

➤ **Extensibility:** It is the ability of smooth expansion of the system by adding processing and storage power to the system. Ideally, the parallel database system should provide two extensibility advantages:

- **Linear Speedup** and
- **Linear Scaleup.**

Linear Speedup refers to a linear increase in performance for a constant database size and linear increase in processing and storage power.

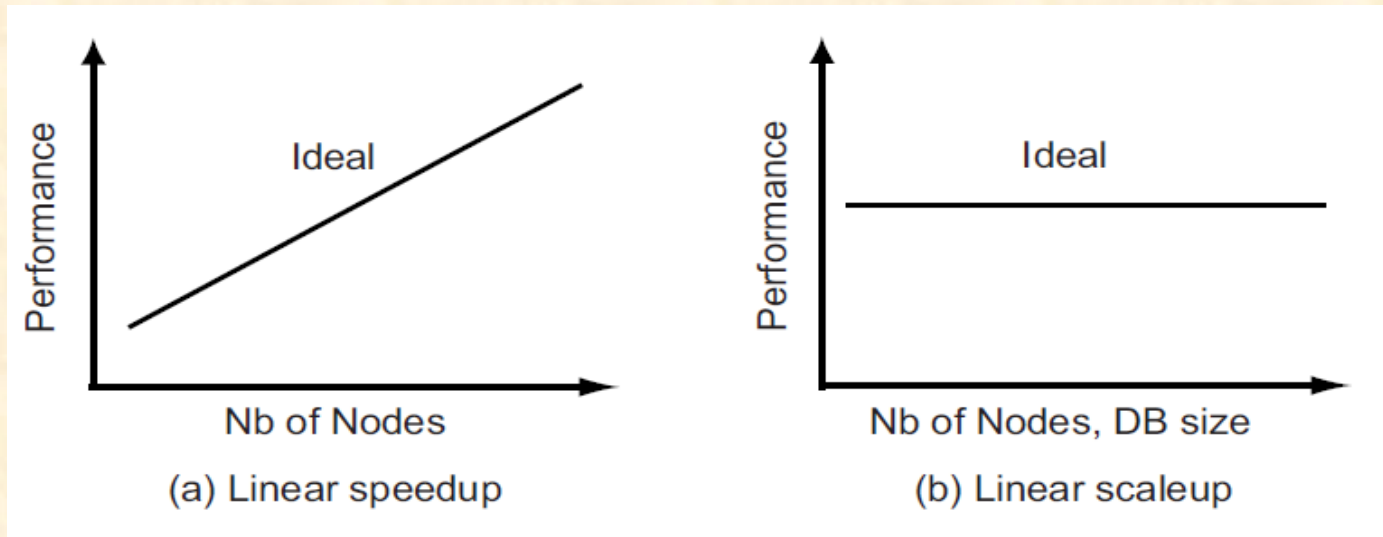
Linear Scaleup refers to a sustained performance for a linear increase in both database size and processing and storage power.



Parallel DBMS

❑ Functional Aspects provided by parallel database systems

➤ Linear speedup and Linear scaleup.





Parallel DBMS

❑ Complications of Parallel DBMS

➤ Cost:

- ✓ Parallel DBMS needs lots of processors and disks
- ✓ It is not easy to implement parallel DBMSs.

➤ Resources:

- ✓ It is not easy to keep up with modern, cheap, and efficient resources. Implementing DBMS requires the users or the company to have renewal of resources, changing or maintaining resources, or even replacement of resources.

➤ Difficulty of Managing Systems:

- ✓ It is not easy to manage the system.
- ✓ When there needs to be a software update, a replacement, or maintenance that all of the system needs to do, it will be time consuming and resource consuming.



Parallel DBMS

❑ Difference between Parallel and Distributed databases

❑ Parallel DBMS: Features

- ✓ A parallel DBMS is a DBMS that runs across multiple processors and is designed to execute operations in parallel,
- ✓ There are parallel working of CPUs.
- ✓ It improves performance.
- ✓ It divides large tasks into various other tasks.
- ✓ Completes works very quickly.
- ✓ Processes are tightly coupled and constitutes a single database system i.e., the parallel database is a centralized database and data reside in a single location.
- ✓ Query processing and transaction is complicated..
- ✓ Data is partitioned among various disks so that it can be retrieved faster.



Parallel DBMS

❑ Difference between Parallel and Distributed databases

❑ Parallel DBMS: Features

- ✓ There are 3 types of architecture: shared memory, shared disk, and shared shared-nothing.
- ✓ Query optimization is more complicated.
- ✓ Data is generally not copied.
- ✓ Parallel databases are generally homogeneous in nature.
- ✓ Skew is the major issue with the increasing degree of parallelism in parallel databases.



Parallel DBMS

❑ Difference between Parallel and Distributed databases

❑ Distributed DBMS: Features

- ✓ A Distributed database is defined as a logically related collection of data that is shared which is physically distributed over a computer network on different sites.
- ✓ It is a group of logically related shared data
- ✓ The data gets split into various fragments
- ✓ There may be a replication of fragments
- ✓ The sites are linked by a communication network
- ✓ The sites are loosely coupled and share no physical components.
- ✓ Query processing and transaction is more complicated.



Parallel DBMS

❑ Difference between Parallel and Distributed databases

❑ Distributed DBMS: Features

- ✓ Each site preserve a local database system for faster processing due to the slow interconnection between sites
- ✓ Distributed databases are generally a kind of shared-nothing architecture
- ✓ In distributed databases, query Optimization techniques may be different at different sites and are easy to maintain
- ✓ Data is replicated at any number of sites to improve the performance of systems
- ✓ Distributed databases may be homogeneous or heterogeneous in nature
- ✓ Blocking due to site failure and transparency are the major issues in distributed databases.



Parallel DBMS

□ Design of Parallel Databases

- A parallel DBMS is a DBMS that runs across multiple processors or CPUs and is mainly designed to execute query operations in parallel.
- In Parallel Databases, mainly there are three architectural designs for parallel DBMS. They are as follows:

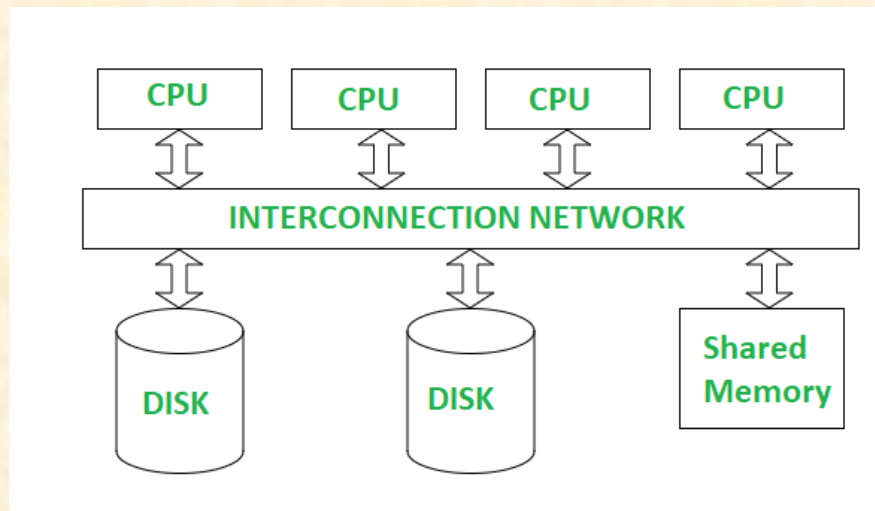
- I. Shared Memory Architecture**
- II. Shared Disk Architecture**
- III. Shared Nothing Architecture**



Parallel DBMS

❑ Shared Memory Architecture

- There are multiple CPUs that are attached to an interconnection network.
- They are able to share a single or global main memory and common disk arrays.
- Any processor has access to any memory module or disk unit through a fast interconnect.
- All the processors are under the control of a single operating system.





Parallel DBMS

❑ Shared Memory Architecture

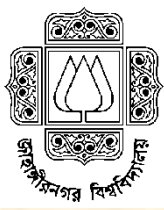
Advantages:

- ✓ Simplicity and easier load balancing
- ✓ It has high-speed data access for a limited number of processors.
- ✓ The communication is efficient.

Disadvantages:

- ✓ It cannot use beyond 80 or 100 CPUs in parallel.
- ✓ The bus or the interconnection network gets block due to the increment of the large number of CPUs.
- ✓ High cost, limited extensibility and low availability.

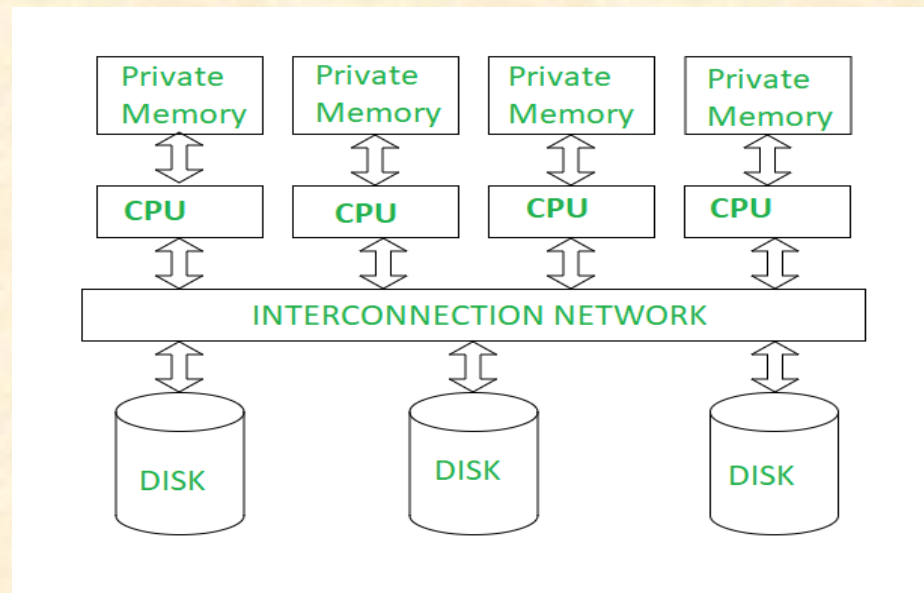
Example: XPRS, DBS3, and Volcano.

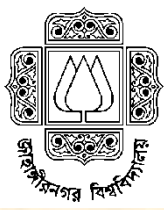


Parallel DBMS

❑ Shared Disk Architecture

- Various CPUs are attached to an interconnection network.
- Each CPU has its own memory and all of them have access to the same disk.
- The memory is not shared among CPUs, therefore each node has its own copy of the operating system and DBMS.
- Shared disk architecture is a loosely coupled architecture optimized for applications that are inherently centralized.





Parallel DBMS

❑ Shared Disk Architecture

Advantages:

- ✓ The interconnection network is no longer a bottleneck each CPU has its own memory.
- ✓ Load-balancing is easier in shared disk architecture.
- ✓ There is better fault tolerance.
- ✓ High extensibility and easy migration from centralized systems.

Disadvantages:

- ✓ If the number of CPUs increases, the problems of interference and memory contentions also increase.
- ✓ There's also exists a scalability problem.
- ✓ Higher complexity

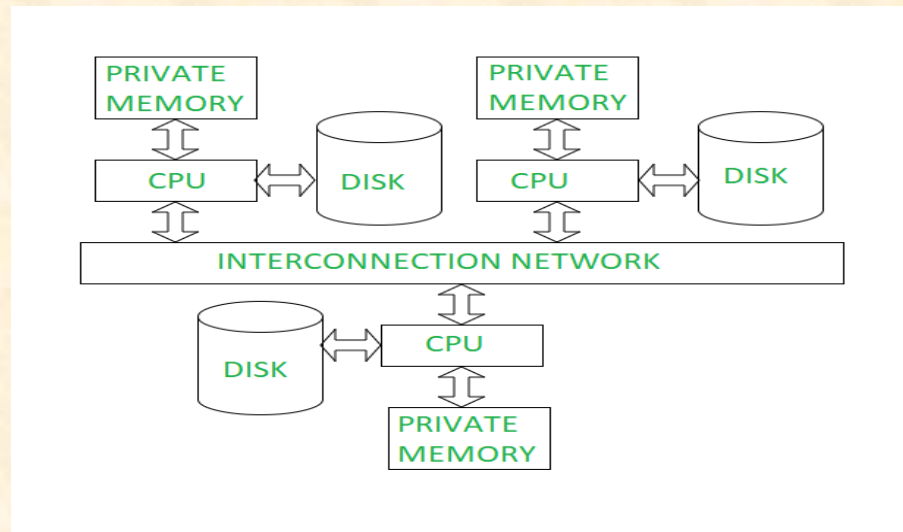
Example: IBM's IMS/VS and DEC's VAX DBMS.



Parallel DBMS

❑ Shared Nothing Architecture

- Multiple processor architecture in which each processor has its own memory and disk storage.
- No two CPUs can access the same disk area. In this architecture, no sharing of memory or disk resources is done. It is also known as **Massively parallel processing (MPP)**.
- Similar to shared-disk, each processor is under the control of its own copy of the operating system. Then, each node can be viewed as a local site in a distributed database system





Parallel DBMS

❑ Shared Nothing Architecture

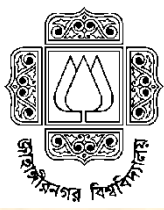
Advantages:

- ✓ It has better scalability as no sharing of resources is done
- ✓ Multiple CPUs can be added
- ✓ Lower cost, high extensibility, and high availability.

Disadvantages:

- ✓ Much more complex to manage
- ✓ The cost of communications is higher as it involves sending of data and software interaction at both ends
- ✓ The cost of non-local disk access is higher than the cost of shared disk architectures.

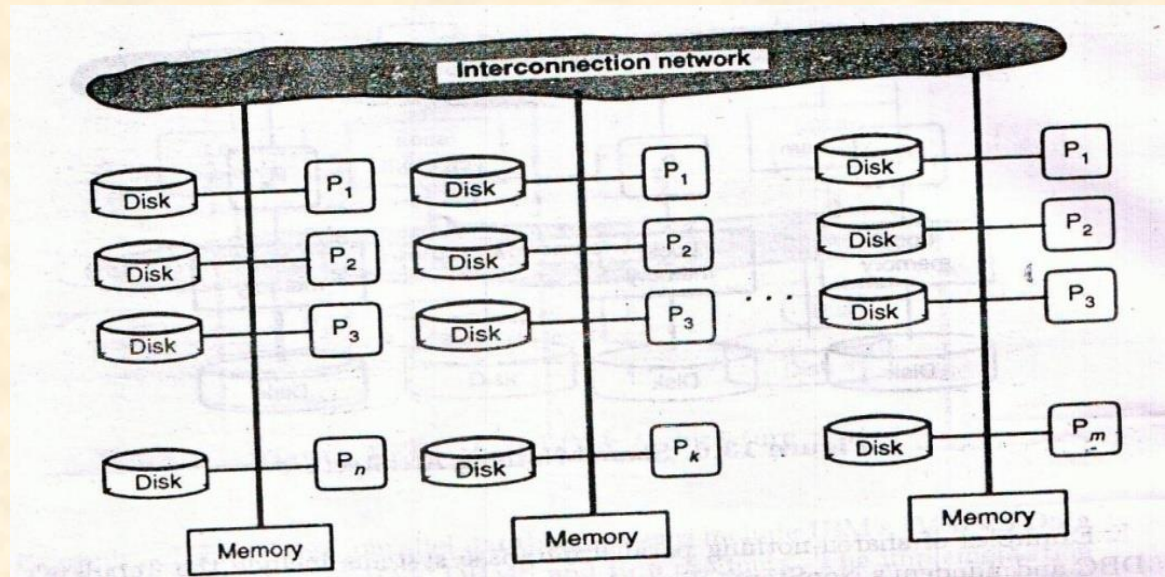
Example: BUBBA, EDS, GAMMA, GRACE, and PRISMA.



Parallel DBMS

❑ Hierarchical Architecture (Hybrid architecture)

- This architecture is a combination of shared disk, shared memory and shared nothing architectures.
- This architecture is scalable due to availability of more memory and many processors.





Parallel DBMS

❑ Hierarchical Architecture (Hybrid architecture)

Advantages:

- ✓ Load balancing is easy
- ✓ Increases performance

Disadvantages:

- ✓ Costly compared to other architectures.
- ✓ Limited number of large nodes, e.g., 4 x 16 processor nodes
- ✓ High number of small nodes, e.g., 16 x 4 processor nodes.

Example: NCR's Teradata



Parallel DBMS

❑ Components of Parallel DBM Architecture

It has three major components or subsystems.

Session Manager: It performs the connections and disconnections between the client processes and the two other subsystems.

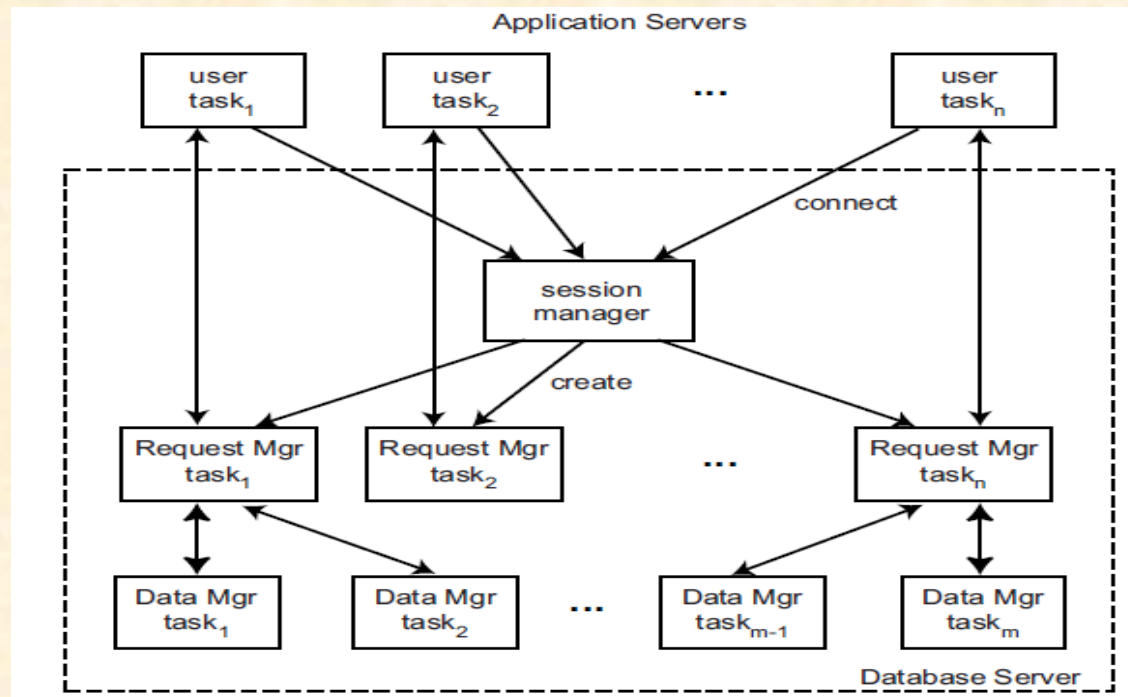
Transaction Manager: It receives client transactions related to query compilation and execution. It can access the database directory that holds all meta-information about data and programs. Depending on the transaction, it activates the various compilation phases, triggers query execution, and returns the results as well as error codes to the client application.



Parallel DBMS

❑ Components of Parallel DBM Architecture

Data Manager: It provides all the low-level functions needed to run compiled queries in parallel.





Parallel DBMS

❑ Types of Database Parallelism

- Parallelism is used to support speedup, where queries are executed faster because more resources, such as processors and disks, are provided.
- Parallelism is also used to provide scale-up, where increasing workloads are managed without increase response-time, via an increase in the degree of parallelism.
- We can achieve parallelism in a query by the following methods :

I/O parallelism

Intra-query parallelism

Inter-query parallelism

Intra-operation parallelism

Inter-operation parallelism



Parallel DBMS

❑ I/O Parallelism (Data Partitioning)

- It is a form of parallelism in which the relations are partitioned on multiple disks to reduce the retrieval time of relations from the disk.
- The data inputted is partitioned and then processing is done in parallel with each partition.
- The results are merged after processing all the partitioned data.
- It is also known as **Data-Partitioning**.
- We have four types of data partitioning in I/O parallelism:
 - i. **Round-robin**
 - ii. **Hash**
 - iii. **Range partitioning and**
 - iv. **Schema partitioning**



Parallel DBMS

❑ I/O Parallelism (Data Partitioning)

Round-robin:

- ✓ Round-robin partitioning is the simplest strategy.
- ✓ It ensures uniform data distribution.
- ✓ With n partitions, the i th tuple in insertion order is assigned to partition $(i \bmod n)$.

Hash:

- ✓ **Hash partitioning** applies a hash function to some attribute that yields the partition number.
- ✓ This strategy allows exact-match queries on the selection attribute to be processed by exactly one node and all other queries to be processed by all the nodes in parallel.
- ✓ For example, there are 4 disks *disk1*, *disk2*, *disk3*, and *disk4* through which the data is to be partitioned. Now if the hash function returns 3, then the row is placed on *disk3*.

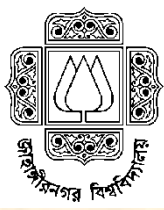


Parallel DBMS

❑ I/O Parallelism (Data Partitioning)

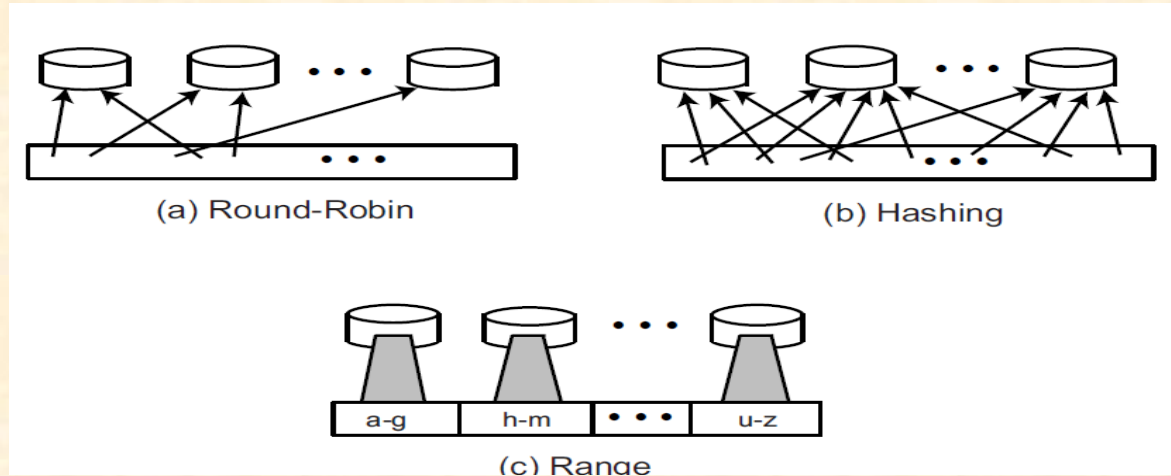
Range partitioning:

- ✓ **Range partitioning** distributes tuples based on the value intervals of some attribute.
- ✓ It is well-suited for range queries.
- ✓ However, range partitioning can result in high variation in partition size.
- ✓ For example, we have 3 disks numbered 0, 1, and 2 in range partitioning, and may assign relation with a value that is less than 5 to disk0, values between 5-40 to disk1, and values that are greater than 40 to disk2.



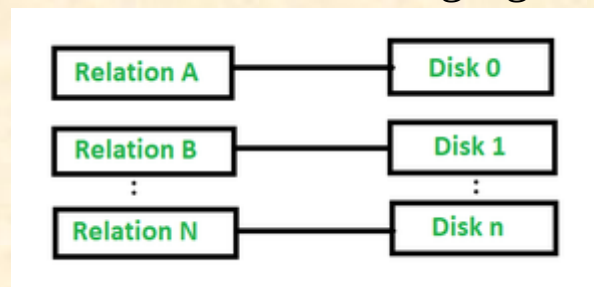
Parallel DBMS

❑ I/O Parallelism (Data Partitioning)



Schema partitioning:

➤ In schema partitioning, different tables within a database are placed on different disks. As shown in the following figure:





Parallel DBMS

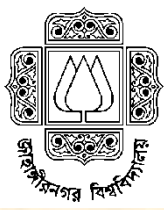
❑ I/O Parallelism (Data Partitioning)

Horizontal Parallelism:

It means that the database is partitioned across multiple disks, and parallel processing occurs within a specific task that is performed concurrently on different processors against different sets of data.

Vertical Parallelism:

It occurs among various tasks. All component query operations (i.e., scan, join, and sort) are executed in parallel in a pipelined fashion.



Parallel DBMS

❑ Parallelism in Query

There are two types of parallelism in query execution in DBMS:

- Intra-query parallelism and
- Inter-query parallelism.

Intra-query Parallelism

- ✓ This is an efficient and powerful technique that breaks down a single query into multiple smaller tasks, which are then executed simultaneously on multiple processors and disks.
- ✓ A simple example— a voluminous database requiring sorting data based on certain parameters. Instead of executing this as one big task (which could be time-consuming), intra-query parallelism divides it into manageable sub-tasks implemented concurrently.
- ✓ The results from these individual operations are then combined to produce the final output.
- ✓ This method significantly boosts the speed of query execution by distributing processing power across different CPUs.



Parallel DBMS

❑ Parallelism in Query

Inter-query Parallelism

- ✓ Different queries or transactions are executed in parallel.
- ✓ It increases overall system throughput and reduces query response time.
- ✓ It makes more efficient use of system resources.
- ✓ Thus, the primary use of inter-query parallelism is to scale up a transaction processing system to support a more significant number of transactions per second.
- ✓ For example, if one query is performing a long-running operation, such as a complex join, other queries can still be processed independently without being blocked.



Parallel DBMS

❑ Parallelism in Query

Inter-query Parallelism

- ✓ **For example:** If there are 6 queries, each query will take 3 seconds for evaluation. Thus, the total time taken to complete evaluation process is 18 seconds. Inter query parallelism achieves this task only in 3 seconds.
- ✓ Implementing interquery parallelism involves various techniques, such as dividing the workload into smaller tasks that can be executed simultaneously across different processors.
- ✓ Parallel database systems employ sophisticated algorithms for distributing workloads effectively.



Parallel DBMS

❑ Intra-operation Parallelism

✓ Intra-operation parallelism is a sort of parallelism in which we parallelize the execution of each individual operation of a task like sorting, joins, projections, and so on.

✓ Let's take an SQL query example:

```
SELECT * FROM Vehicles ORDER BY Model_Number;
```

✓ In the above query, the relational operation is sorting and since a relation can have a large number of records in it, the operation can be performed on different subsets of the relation in multiple processors, which reduces the time required to sort the data.

❑ Inter-operation Parallelism

✓ When different operations in a query expression are executed in parallel, then it is called inter-operation parallelism.



Parallel DBMS

❑ Skew in Parallel DB

- The distribution of tuples onto disks may be uneven or skewed.
- Some disks may have few tuples, while others may have many tuples.

❑ Types of Skew

Attribute-value Skew:

- ✓ Some values may have appeared in the partitioning attributes of many tuples.
- ✓ All those tuples that have the same value for the partitioning attribute are placed in the same partition.
- ✓ Such situation can occur with range-partitioning and hash-partitioning.



Parallel DBMS

□ Types of Skew

Partition Skew:

- ✓ With the range-partitioning technique, a badly chosen partition vector may assign too many tuples to some partitions and too few tuples to the other partitions.
- ✓ This is less likely with hash-partitioning technique if a good hash-function is chosen.



Parallel DBMS

❑ Handling Skew in Range-Partitioning

Assume that the partitioning attribute forms a key of the relation. For creating a **balanced partitioning vector**:

- ✓ Sort the entire relation on the partitioning attribute field.
- ✓ Construct the partition vector by scanning the relation in sorted order as follows:

After every $1/n^{th}$ of the relation has been read, the value of the partitioning attribute of the next tuple is added to the partition vector.

Here, n denotes the number of partitions to be constructed.

- ✓ Duplicate entries or imbalances can result if duplicates are present in partitioning attributes.



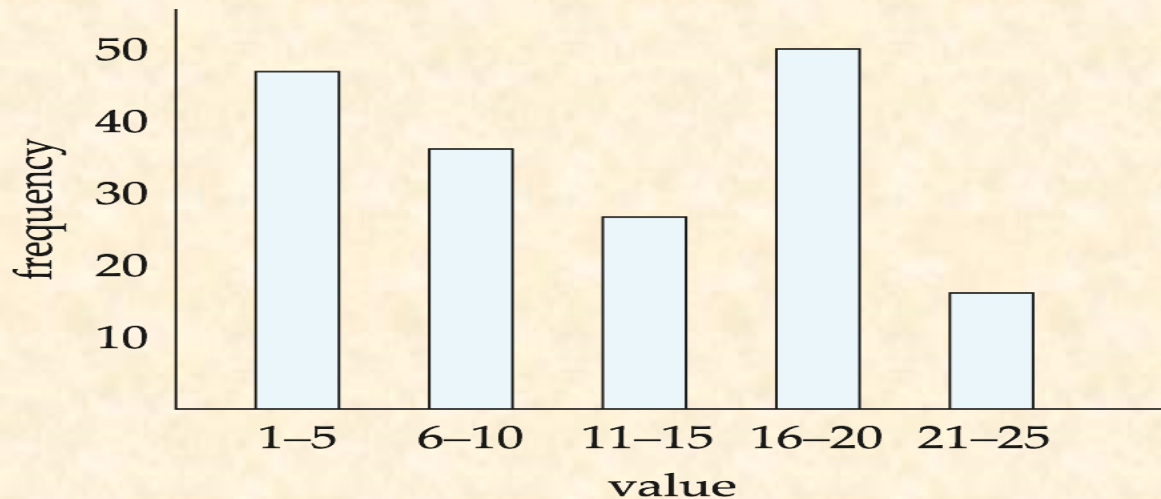
Parallel DBMS

□ Handling Skew using Histograms

➤ Balanced partitioning vector can be constructed from histogram in a relatively straightforward fashion.

Assume uniform distribution within each range of the histogram

➤ Histogram can be constructed by scanning relation.





Parallel DBMS

□ Query Processing

Query Processing includes:

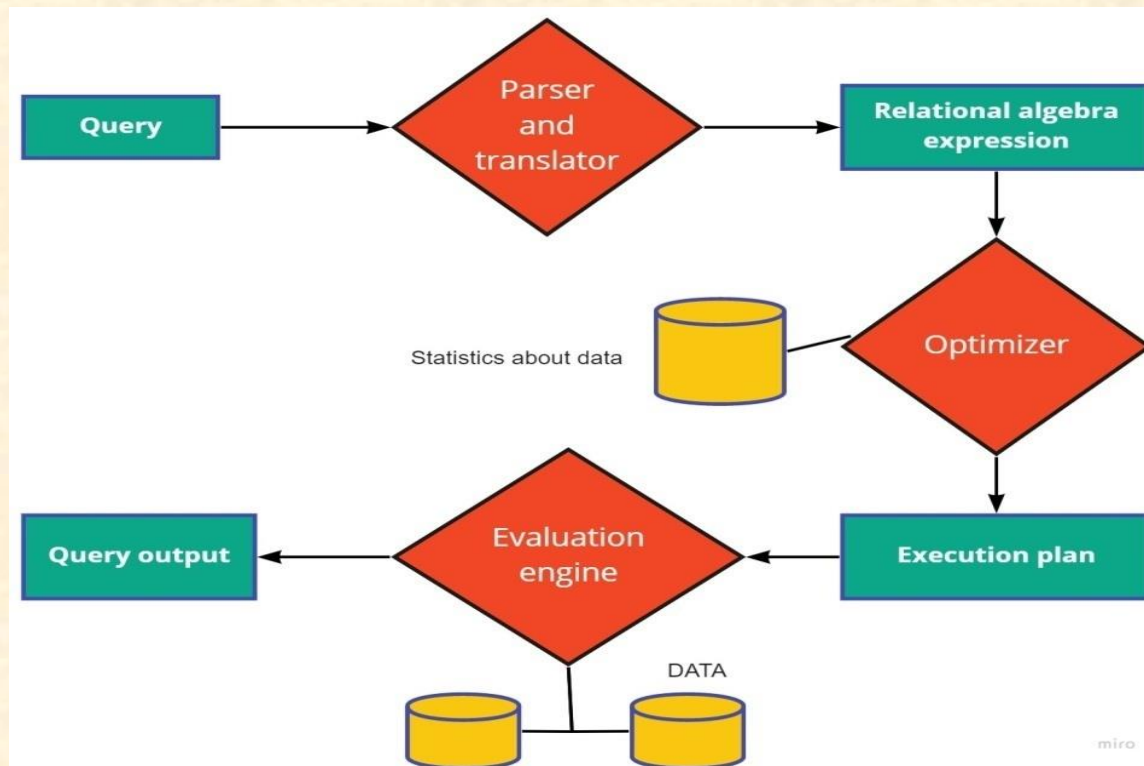
- Conversion of queries written in a high-level language such as SQL into a form that can be understood by the physical level implementation of the database,
- Query optimization techniques, and
- Evaluation of the query.



Parallel DBMS

❑ Query Processing

Steps involved in query processing:





Parallel DBMS

□ Query Processing

Parser and Translator:

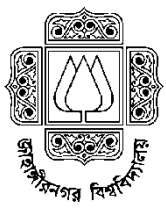
- ✓ The first step in query processing is parsing and translation. Parser checks the syntax of the query whether the relations mentioned are present in the database or not.
- ✓ A high-level query language such as SQL is suitable for human use. But, it is totally unsuitable to system internal representation. Therefore, translation is required. The internal representation can be extended form of relational algebra.
- ✓ A parse tree of the query is constructed and then translated into relational algebraic expression.

Example: consider the following SQL query:

```
SELECT s.name
FROM reserves R, sailors S
Where R.sid=S.sid
      AND R.bid=100 And S.rating>5
```

Its corresponding relational algebraic expression:

$$\Pi_{s.name} (\sigma_{R.bid=100 \wedge S.rating > 5} (R \bowtie_{R.sid=S.sid} S))$$



Parallel DBMS

❑ Query Processing

Optimizer:

- ✓ A SQL query can be written in many different ways. A Query can also have different corresponding relational algebra expressions
- ✓ The task of the query optimizer is to assess all possible ways to execute a query and choose the most effective one. This is done during the optimization process.
- ✓ An optimized query also depends on how the data is stored in the file organization.
- ✓ Query optimization is the process of selecting an efficient execution plan for evaluating the query.
- ✓ After parsing of the query, parsed query is passed to query optimizer, which generates different execution plans to evaluate parsed query and select the plan with least estimated cost.
- ✓ Catalog manager helps optimizer to choose best plan to execute query generating cost of each plan.

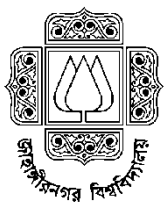


Parallel DBMS

❑ Query Processing

Execution plan:

- ✓ Different evaluation plans for a particular query have different query costs.
- ✓ The cost may include the number of disk accesses, CPU time for execution of the query, time of communication in the case of distributed databases.



Parallel DBMS

❑ Query Processing

Basic Operation:

Usually, user queries are submitted to DBMS as SQL queries. During the parsing and translation phase, the given query is translated into its internal form. In generating the internal form of the query, the parser checks the syntax of the user's query, verifies that the relation names appearing in the query are names of the relations in the database and so on. The system constructs a parse tree representation of the query, which it then translates into a relational algebra expression.

After parsing and translation into relational algebra expression, the query is then transformed into a form which is usually query tree or graph that can be handled by the optimization engine. Query representation during the optimization phase, the optimization engine performs various analyses on the query data. It applies various rules to the internal data structures of the query to transform these structures into equivalent and efficient representation. It then generates valid evaluation plans based upon the rules applied. From the generated evaluation plans, the best evaluation plan to be executed is determined and passed onto the query execution engine. The final phase in processing a query is the evaluation phase. During the evaluation phase, the best evaluation plan generated by the optimization engine is selected and then executed.



Parallel DBMS

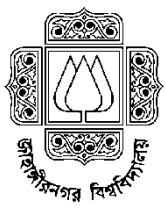
□ Parallel Query Optimization

- Parallel query optimization refers to the process of producing an **execution plan** for a given query that minimizes an objective cost function. The selected plan is the best one within a set of candidate plans examined by the **optimizer**.
- Parallel query optimization exhibits similarities with distributed query processing. However, it takes advantage of both intra-operator parallelism and inter-operator parallelism.
- A parallel query optimizer can be seen as three components: **a search space, a cost model, and a search strategy**.

The search space is the set of alternative execution plans to represent the input query.

The cost model predicts the cost of the given execution plan.

The search strategy explores the search space and selects the best plan.



Parallel DBMS

❑ How can we optimize a SQL query?

Optimizing an SQL query involves various techniques to improve its performance and efficiency. Some common strategies include:

- ✓ Indexing: Create appropriate indexes on columns used in WHERE, JOIN, and ORDER BY clauses to speed up data retrieval.
- ✓ Filtering: Use efficient WHERE clauses to minimize the number of rows processed.
- ✓ Avoid SELECT *: Retrieve only the necessary columns to reduce data transfer.
- ✓ JOIN Optimization: Choose the most efficient JOIN type (INNER JOIN, LEFT JOIN, etc.) and ensure relevant columns are indexed.
- ✓ Subqueries: Optimize subqueries to avoid excessive data processing.
- ✓ Use EXISTS instead of IN: Replace IN clauses with EXISTS for better performance.
- ✓ Limit Data: Use LIMIT or TOP to restrict the number of returned rows, especially for large result sets.
- ✓ Caching: Utilize query caching mechanisms provided by the database system.
- ✓ Review Execution Plan: Analyze the query execution plan and optimize it if necessary using query hints.
- ✓ Partitioning: If applicable, partition large tables to improve data retrieval for specific ranges.



Parallel DBMS

❑ What is query optimization process?

- The query optimization process involves analyzing SQL queries to improve their execution efficiency.
- It includes steps such as parsing the query, generating candidate execution plans, estimating costs for each plan, and selecting the most optimal plan based on cost estimations.
- Indexing, query rewriting, and other techniques are applied to reduce query execution time and improve overall database performance.

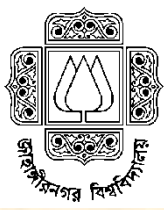


Parallel DBMS

□ Database Integration Process

Database Integration involves the process by which information from participating database can be conceptually integrated to form a single cohesive definition of a multi-database. In other words, it is the process of designing the Global Conceptual Schema (GCS). Database Integration occurs in two general steps: schema translation and schema integration.

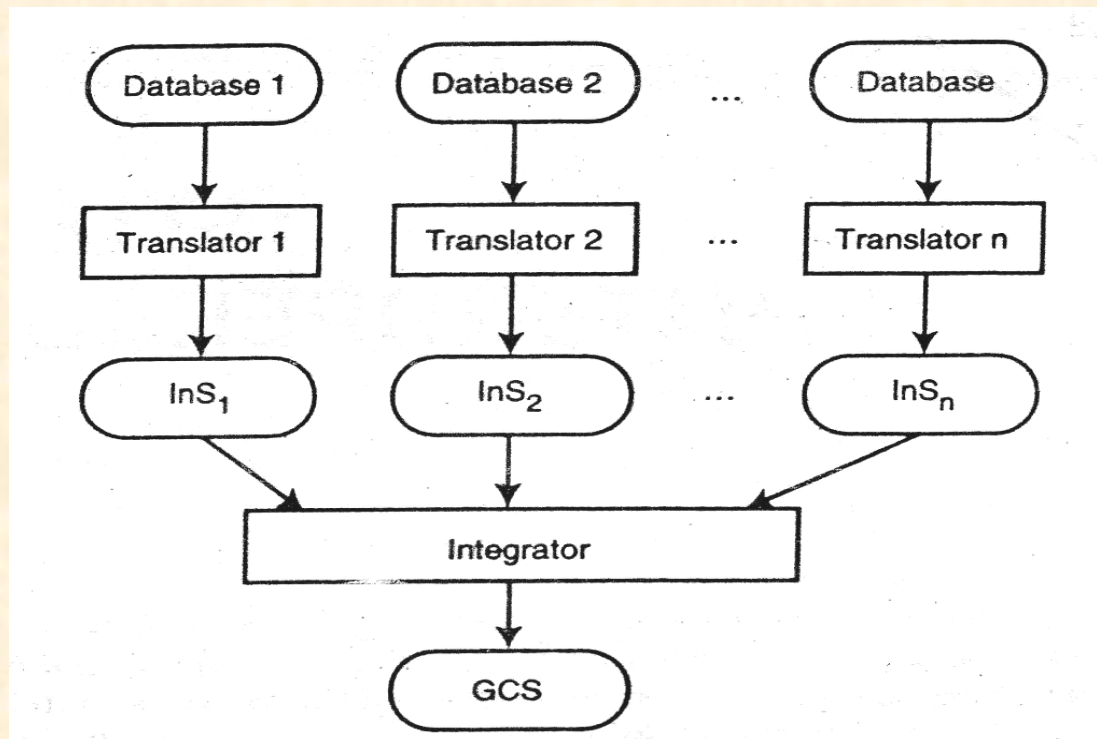
In the first step, the component database schemas are translated to a common intermediate canonical representation ($InS_1; InS_2; : : : ; InS_n$). Clearly, the translation step is necessary only if the component databases are heterogeneous and local schemas are defined using different data models.



Parallel DBMS

❑ Database Integration Process

In the second step, the intermediate schemas are used to generate a GCS. In some methodologies, local external schemas are considered for integration rather than full database schemas.





Parallel DBMS

❑ Database Integration Process

Schema Translation

Schema Translation is the task of mapping from one schema to another. This requires the specification of a target data model for the global conceptual schema definition. Schema translation may not be necessary in a heterogeneous database if it can be accomplished during integration stage. Combining the translation and integration steps provides the integrator with all the information about the entire global database at one time.



Parallel DBMS

❑ Database Integration Process

Schema Integration

Schema Integration is the process of identifying the components of a database which are related to one another, selecting the best representation for the global conceptual schema and finally, integrating the components of each intermediate schema.

Integration methodologies can be classified as binary or n-ary mechanisms. Binary integration methodologies involve the manipulation of two schemas at a time. These can occur in a stepwise (ladder) fashion where intermediate schemas are created for integration with subsequent schemas or in a purely binary fashion, where each schema is integrated with one other, creating an intermediate schema for integration with other intermediate schemas and N-ary integration mechanisms integrate more than two schemas at each iteration.